

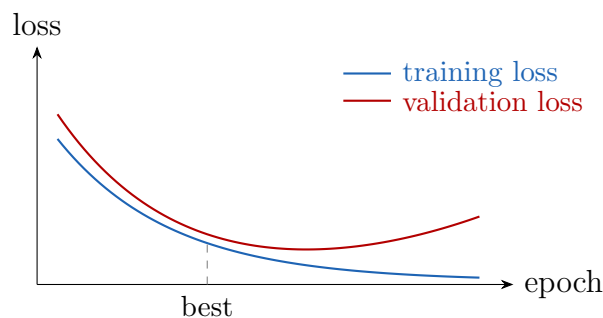
Section 4.1: Regularization

L1 and L2 — Teaching the Network to Generalise

AI Class — Lecture Notes

1 The Problem: Overfitting

A network trained long enough on a small dataset will eventually **memorise** the training examples rather than learning the underlying pattern. It achieves near-zero training loss but fails badly on new data.



The validation loss initially decreases together with the training loss, then starts rising — the network is now fitting noise. **Regularisation** discourages this by penalising large weights, which are the symptom of overfitting.

2 L2 Regularisation (Weight Decay)

Add the sum of squared weights to the loss:

$$J_{\text{reg}} = J + \lambda \sum_i w_i^2$$

where $\lambda > 0$ controls how strongly large weights are penalised.

2.1 Effect on the Gradient

Taking the partial derivative:

$$\frac{\partial J_{\text{reg}}}{\partial w} = \frac{\partial J}{\partial w} + 2\lambda w$$

The gradient descent update becomes:

$$w \leftarrow w - \eta \left(\frac{\partial J}{\partial w} + 2\lambda w \right)$$

Rearranging:

$$w \leftarrow \underbrace{(1 - 2\eta\lambda)}_{\text{shrink factor}} w - \eta \frac{\partial J}{\partial w}$$

Every update *shrinks* the weight slightly toward zero before applying the gradient step. This is why L2 regularisation is also called **weight decay**.

2.2 Hand-Calculated Example

Using $w_1^{(2)} = 0.5$, gradient $g = -0.084$ (from Section 3.4), $\eta = 0.1$, and $\lambda = 0.01$:

Without regularisation

$$w \leftarrow 0.5 - 0.1 \times (-0.084) = 0.5084$$

With L2 regularisation

$$\frac{\partial J_{\text{reg}}}{\partial w} = -0.084 + 2(0.01)(0.5) = -0.084 + 0.010 = -0.074$$

$$w \leftarrow 0.5 - 0.1 \times (-0.074) = 0.5074$$

The regularised update is slightly smaller (0.5074 vs 0.5084): the penalty pulled the weight back toward zero by 0.001. Small per step, but it accumulates over thousands of updates.

3 L1 Regularisation (Sparsity)

Replace the squared penalty with the *absolute value*:

$$J_{\text{reg}} = J + \lambda \sum_i |w_i|$$

3.1 Effect on the Gradient

$$\frac{\partial J_{\text{reg}}}{\partial w} = \frac{\partial J}{\partial w} + \lambda \text{sign}(w)$$

where $\text{sign}(w) = +1$ if $w > 0$, and -1 if $w < 0$.

Unlike L2, the penalty is **constant** — it does not scale with $|w|$. This means L1 applies the same pressure to a weight of 0.5 as to a weight of 0.001, and will push even tiny weights all the way to zero.

3.2 L1 vs L2: The Key Difference

The distinction shows up clearly when comparing a large weight and a small weight under the same conditions ($g = -0.084$, $\lambda = 0.01$):

Weight	Method	$\partial J_{\text{reg}}/\partial w$	$w_{\text{new}} (\eta = 0.1)$
$w = 0.5$	No reg	-0.084	0.508
	L2	$-0.084 + 2(0.01)(0.5) = -0.074$	0.507
	L1	$-0.084 + 0.01(+1) = -0.074$	0.507
$w = 0.01$	No reg	-0.084	0.018
	L2	$-0.084 + 2(0.01)(0.01) = -0.0838$	0.019
	L1	$-0.084 + 0.01(+1) = -0.074$	0.017

For the large weight ($w = 0.5$), L1 and L2 give almost identical results. For the small weight ($w = 0.01$), L2 barely penalises it (penalty = 0.0002) while L1 applies full pressure (penalty = 0.01) — driving it toward zero.

When to use which: Use **L2** when you expect many small weights to all contribute. Use **L1** when you expect only a few weights to matter and want the rest to vanish completely (sparse solution). In practice L2 is the default; L1 is common in feature selection.

4 Choosing λ

λ	Effect	Symptom
Too small	Almost no regularisation	Validation loss diverges from training loss
Good	Validation loss tracks training loss	Model generalises well
Too large	Weights shrink to near zero	Both training and validation loss stay high

Typical starting values: $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$. Tune by monitoring the gap between training and validation loss.

PyTorch. L2 regularisation is built into every PyTorch optimiser as `weight_decay`. See `code_examples.py` for usage. L1 is added manually to the loss; an example is also in the code file.